

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №4

Выполнил:

Малышенко А. Р.

Группа К34392

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Содержание отчета

1) Постановка задачи	3
2) Ход работы	3
3) Вывод	7

1) Постановка задачи

Необходимо упаковать ваше приложение в docker-контейнеры и обеспечить сетевое взаимодействие между различными частями вашего приложения, а также настроить общение микросервисов между собой посредством RabbitMQ. Делать это можно как с помощью docker-compose так и с помощью docker swarm. При разумном использовании swirl вы получите дополнительные баллы.

2) Ход работы

Для начала необходимо добавить к каждому микросервису dockerfile (рисунок 1).

WORKDIR – указывает путь папки внутри контейнера.

COPY package*.json – копирует одноименный файл с зависимостями в контейнер

RUN npm install – устанавливает эти зависимости внутри контейнера.

COPY . . – копирует остальные файлы проекта в контейнер.

Далее запускается сервис на порте (в примере 3000). Для всех сервисов docker файл похож, меняется только порт.

```
FROM node:18

WORKDIR /usr/src/monolith

COPY package*.json ./

RUN npm install
RUN npm uninstall bcrypt
RUN npm install bcrypt

COPY . .

RUN npm run build

EXPOSE 3000

CMD [ "node", "dist/index.js" ]
```

Рисунок 1 – Dockerfile

Далее необходимо написать docker compose, который будет запускать эти контейнеры вместе (рисунки 2, 3, 4).

```
networks:
  app-network:
    driver: bridge

services:
  app-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: lab4
      PGDATA: /var/lib/postgresql/data/pgdata
    volumes:
      - ./volumes/app-db:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready" ]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  monolith:
    build:
      context: ./monolith
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    depends_on:
      app-db:
        condition: service_healthy
    environment:
      db: postgresql://postgres:admin@app-db:5432/lab4
      host: localhost
      port: 3000
      api_version: v1
      authUrl: http://user_auth_service:9191/user_auth_service/v1/
      infoUrl: http://user_info_service:9595/user_info_service/v1/
```

Рисунок 2 – docker compose

```

volumes:
  - ./monolith:/usr/src/monolith:cached
  - /usr/src/monolith/node_modules
networks:
  - app-network

user_info_service:
  build:
    context: ./user_info_service
    dockerfile: Dockerfile
  ports:
    - "9595:9090"
  depends_on:
    app-db:
      condition: service_healthy
    monolith:
      condition: service_started
  environment:
    db: postgresql://postgres:admin@app-db:5432/lab4
    host: localhost
    port: 9595
    api_version: v1
  volumes:
    - ./user_info_service:/usr/src/user_info_service:cached
    - /usr/src/user_info_service/node_modules
  networks:
    - app-network

user_auth_service:
  build:
    context: ./user_auth_service
    dockerfile: Dockerfile
  ports:
    - "9191:9090"
  depends_on:
    app-db:
      condition: service_healthy
    monolith:
      condition: service_started

```

Рисунок 3 – docker compose

```

environment:
  db: postgresql://postgres:admin@app-db:5432/lab4
  host: localhost
  port: 9191
  api_version: v1
volumes:
  - ./user_auth_service:/usr/src/user_auth_service:cached
  - /usr/src/user_auth_service/node_modules
networks:
  - app-network

```

Рисунок 4 – docker compose

Теперь необходимо запустить docker compose командой в консоли
docker compose up (рисунок 5).

```

PS E:\study\23-24\lab4> docker compose up
time="2024-10-02T13:08:39+03:00" level=warning msg="E:\\study\\23-24\\lab4\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 4/0
 ✓ Container lab4-app-db-1          Created
 ✓ Container lab4-mono1ith-1        Created
 ✓ Container lab4-user_auth_service-1 Created
 ✓ Container lab4-user_info_service-1 Created
Attaching to app-db-1, mono1ith-1, user_auth_service-1, user_info_service-1

```

Рисунок 5 – запуск docker compose

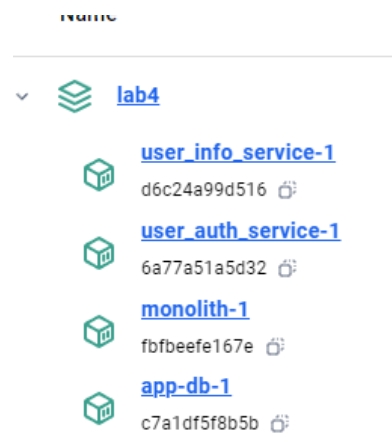


Рисунок 6 – все контейнеры работают

Теперь при отправке запроса на регистрацию получаем верный ответ (рисунок 7).

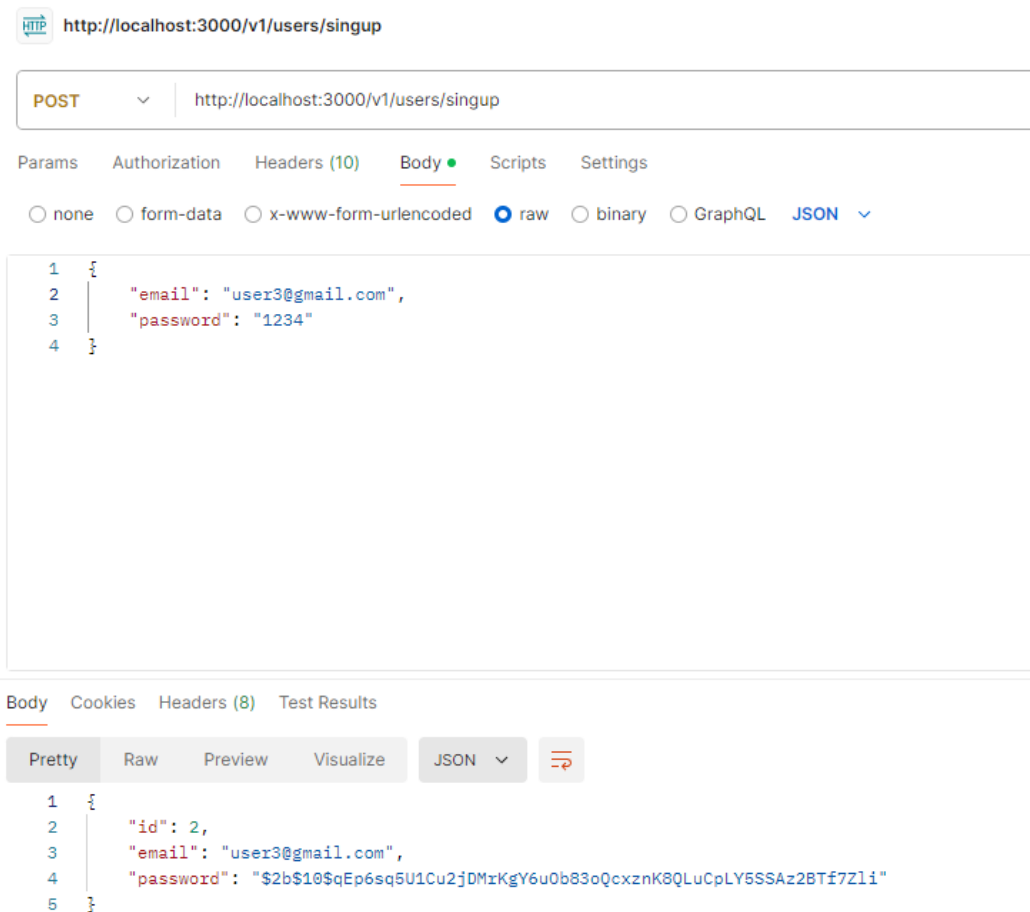


Рисунок 7 – запрос/ответ

3) Вывод

В ходе лабораторной работы №4 была реализована связь между микросервисами и запущены в docker при помощи docker compose.